# Mohar

A sealed custody chain for examination papers

Study guide — basics to advanced

This document explains, from first principles, what the system does, which cryptography it uses and why, how the access engine reaches a decision, and which parts of it are honestly unfinished. Read it end to end and you will be able to defend every design choice in the repository.

PREPARED FOR

Smart India Hackathon — technical presentation and Q&A

SCOPE

Architecture, cryptography, custody model, access control, operations

READING ORDER

Parts 1–4 are foundational. Parts 5–10 are the system proper. Parts 11–12 are for the demo and the questions.

READING TIME

Roughly 70 minutes at a careful pace

# Contents

# 1 The problem Mohar actually solves

### The visible problem

Examination question papers in India travel a long physical road. A paper is set, printed, sealed into bundles, handed to a courier, driven across a district, held overnight in a custodian's strongroom, delivered to an examination centre, stored again, and finally opened minutes before the examination begins. At every handover the bundle changes hands between people who may never meet again.

Leaks happen along this road. When a paper surfaces on a messaging app the night before an examination, an entire cohort's results become unsafe, and the usual remedy is to cancel and re-conduct the examination at enormous public cost.

### The problem underneath it

The instinctive framing is that leaks are a *detection* problem — that we need to catch the leak faster. That framing is wrong, and getting this right is the single most important idea in the whole project.

> **The real failure is evidentiary.** Leaks are usually detected. What collapses afterwards is the case. By the time an investigator asks who held the bundle between 21:40 and 06:15, the answer lives in a paper register with handwritten times, a photograph of a seal on somebody's phone, and four people's recollections. None of that survives cross-examination. A prosecution that cannot pin custody to a person at a time cannot convict.

So Mohar is not a leak detector. It is a machine for producing custody evidence that is admissible, tamper-evident, and complete enough that the gap in the record *is itself* the finding. If a bundle has no recorded holder for eighteen hours, that silence is a fact the system reports, not a fact it hides.

### Restating it as an engineering brief

- **Every custody act must be recorded by the person who performed it**, on a device bound to them, with a signature only that device could produce.
- **The record must be impossible to edit after the fact** — including by the people who run the system.
- **Absence of a record must be as loud as a bad record.** A chain with a hole in it must report the hole.
- **It must work offline**, because strongrooms have concrete walls and rural centres lose signal.
- **Any outside party must be able to verify the record** without being given the papers, the database, or trust in the operator.

## 2 Goals, non-goals, and the one honest claim

### What the system claims

| | |
|---|---|
| Tamper-evidence | Any alteration to a past record breaks a hash chain that anyone can recompute. We do not claim the record cannot be altered; we claim alteration cannot be hidden. |
| Attribution | Every entry carries an Ed25519 signature from an enrolled device, and the device is bound to a person and a centre in reference data. |
| Completeness | Custody state is derived by replaying events. A missing handover produces a visible gap, not a plausible-looking timeline. |
| Independent verification | Daily Merkle roots are published. A third party can prove one event was included without seeing any other event. |
| Least-privilege access | A stage-scoped key that dies after six hours, checked against fifteen independent conditions before any bundle is opened. |

### What the system explicitly does not claim

> **Mohar does not make examinations leak-proof.** A superintendent with legitimate access, a phone, and bad intent can photograph a paper at the moment it is legitimately opened. No custody system can prevent that, and claiming otherwise is the fastest way to lose credibility in a technical review.

What Mohar changes is the cost. It narrows the window in which a leak is possible from days to minutes, and it narrows the set of people who could have done it from hundreds to a named few, with cryptographic evidence of who held what and when. That is a real, defensible, bounded claim. Make exactly that claim and no larger one.

## Threat model in one page

| Adversary | Capability | What stops them |
|---|---|---|
| Opportunistic courier | Physical access in transit; can open and reseal a bundle | Seal serial is registered at seal time and re-read at every hop; a changed serial is an anomaly on the package |
| Colluding custodian | Legitimate possession overnight; wants no record of a detour | Custody gap detection — a stretch of time in transit with no recorded holder is reported as a gap in minutes |
| Insider operator | Database credentials for the running application | The application role holds `INSERT` and `SELECT` only. There is no `UPDATE` or `DELETE` grant on the event table, so the application literally cannot rewrite history |
| Database administrator | Full database access, can rewrite rows directly | Cannot be prevented — but cannot be hidden. Rewriting one row breaks every chain hash after it, and the daily Merkle root already published elsewhere no longer matches |
| Early opener | Wants to open a bundle before the examination window | Custody window check plus key epoch — a key for a stage does not exist outside its window, and the attempt is recorded whether it succeeds or not |
| Key thief | Steals or photographs an issued custody key | The key is stage-scoped, package-scoped, and dies at the end of its six-hour epoch. It is also useless without an enrolled device inside the geofence |

# 3 Cryptographic foundations BASICS

If you already know hashing and signatures, skim to §3.4. Everything here is standard, but the presentation is chosen to make the later design decisions obvious.

## 3.1 A hash function

A cryptographic hash takes any input — a byte, a book, a JSON object — and produces a fixed-length fingerprint. Mohar uses SHA-256, which produces 32 bytes (64 hexadecimal characters).

```
SHA256("Mohar")   → 3b7a...  (32 bytes)
SHA256("Mohar.")  → e91c...  (completely different)
```

Three properties matter, and only these three:

1. **Deterministic.** The same input always gives the same output. This is what lets an auditor recompute our hashes and get our answers.
2. **One-way.** Given the fingerprint you cannot recover the input. This is why we store a *hash* of every custody key rather than the key itself.
3. **Avalanche and collision resistance.** Changing one bit changes roughly half the output bits, and nobody can construct two different inputs with the same fingerprint. This is what makes tampering detectable.

## 3.2 A hash chain

Take a list of records. For each one, compute a hash not just of the record but of the record *plus the previous record's hash*.

```
entry 1:  hash₁ = SHA256( GENESIS  ‖ body₁ )
entry 2:  hash₂ = SHA256( hash₁    ‖ body₂ )
entry 3:  hash₃ = SHA256( hash₂    ‖ body₃ )
entry 4:  hash₄ = SHA256( hash₃    ‖ body₄ )

GENESIS ──▶┌────────┐   ┌────────┐   ┌────────┐   ┌────────┐
           │ body₁  │──▶│ body₂  │──▶│ body₃  │──▶│ body₄  │
           │ hash₁  │   │ hash₂  │   │ hash₃  │   │ hash₄  │
           └────────┘   └────────┘   └────────┘   └────────┘
```

Now suppose someone edits `body₂` — say, changing a handover time to cover an eighteen-hour gap. Recomputing gives a different `hash₂`. But `hash₃` was built from the *old* `hash₂`, so `hash₃` no longer matches. Neither does `hash₄`, or anything after it. To hide the edit the attacker must rewrite every subsequent entry — and if any root has already been published outside the database, even that is not enough.

> **The genesis value.** The first entry has no predecessor, so Mohar uses 32 zero bytes as the `GENESIS` hash. This is a convention, not a cryptographic requirement — it just gives the recursion a defined base case so verification code needs no special first-entry branch.

### 3.3 Digital signatures

A hash proves a record has not changed. It does *not* prove who wrote it — anyone can compute a hash. For attribution we need signatures.

Each enrolled device holds a **private key** it never transmits, and publishes a matching **public key**. Signing produces a value that only the private key could have produced but that anyone with the public key can check.

```
signature = Ed25519_sign(devicePrivateKey, bodyHash)
valid?    = Ed25519_verify(devicePublicKey, bodyHash, signature)
```

Mohar uses **Ed25519** rather than RSA or ECDSA for three practical reasons: signatures are 64 bytes and keys 32 bytes, which matters when a field device queues events offline; signing is deterministic, so there is no catastrophic failure mode from a bad random number generator as ECDSA has; and it is fast enough on cheap Android hardware.

### 3.4 The problem canonical JSON solves

Here is a subtle failure that breaks naïve implementations. A device signs an event body. The server receives it, parses the JSON, stores it, and later re-serialises it to verify the signature. But:

```
{"copies":240,"centre":"JPR-07"}     → hash A
{"centre":"JPR-07","copies":240}     → hash B     ← same data, different hash
{"copies": 240, "centre": "JPR-07"}  → hash C     ← whitespace, different again
```

Semantically identical JSON produces different bytes, therefore different hashes, therefore a signature that fails to verify even though nothing was tampered with. Key order is not preserved across parsers, languages, or database round-trips.

The fix is a **canonicalisation standard**: a single deterministic byte representation for any given JSON value. Mohar uses **RFC 8785 JSON Canonicalization Scheme (JCS)**, via the `canonicalize` package in `packages/crypto-core`. JCS fixes key ordering (lexicographic by UTF-16 code unit), removes insignificant whitespace, and pins number formatting.

> **Say this in the demo.** "We never hash `JSON.stringify` output. We hash RFC 8785 canonical form, so a signature made on an Android device in a strongroom still verifies after the JSON has been through Postgres and back." That one sentence signals you have actually built this rather than sketched it.

# 4 The ledger CORE

## 4.1 What an event is

An event is one custody act: a seal applied, a bundle handed over, a package received, a bundle opened, a key destroyed. It is written once and never modified. Conceptually:

```
{
  seq:           190,                     // monotonic position in the chain
  id:            "…uuid…",
  kind:          "custody.handover",
  occurredAt:    "2026-08-19T21:40:11Z",   // device clock, as reported
  receivedAt:    "2026-08-19T21:44:02Z",   // server clock, on arrival
  clockSkewMs:   231000,                   // the difference, recorded not corrected
  actorPersonId: "…", actorRole: "courier",
  actorDeviceId: "dev-jpr-courier-02",
  cosignDeviceId:"dev-jpr-custodian-01",   // both parties to a handover sign
  lat: 26.9124, lon: 75.7873, geoAccuracyM: 8,
  payload:       { packageId:"JPR-002", fromPersonId:"…", toPersonId:"…",
                   sealSerialRead:"MH-4471-A" },
  bodyHash:      "…",   // SHA256 of canonical body
  prevHash:      "…",   // chain hash of the previous entry
  hash:          "…"    // SHA256(prevHash ‖ bodyHash)
}
```

## 4.2 Why there are two hashes

This trips people up, so be precise about it.

| | `bodyHash` | `hash` (chain hash) |
|---|---|---|
| Computed over | The canonical event body alone | `prevHash ‖ bodyHash` |
| Answers | "Is this event's content unaltered, and who signed it?" | "Is this event in the right place in history?" |
| Depends on | Nothing else in the ledger | Every event before it |
| What signs it | The device's Ed25519 key signs `bodyHash` | Nothing — it is derived, and recomputable by anyone |

The separation matters because it lets you prove a single event to an outsider without handing over the whole chain. Give them the event body and the signature and they verify authorship. Give them the surrounding hashes and an inclusion proof and they verify position. Neither step requires them to see other packages, other centres, or the papers themselves.

## 4.3 Append-only is enforced by the database, not the code

This is the design decision most worth defending in a technical review, so understand the argument properly.

A lot of systems say "append-only" and mean "our code never issues an `UPDATE`". That is a promise, not a control. It survives exactly until someone adds an endpoint, or a maintenance script, or a compromised dependency issues an update on their behalf.

Mohar enforces it one layer down, in Postgres role grants:

```
-- the role the running application connects as
grant select, insert on led.event          to mohar_app;
grant select, insert on led.access_attempt to mohar_app;
--  no UPDATE. no DELETE. no TRUNCATE.

-- a separate role, used only by migrations, holds DDL rights
-- mohar_migrator — never used by the service at runtime
```

> **Why this is stronger.** If an attacker achieves full remote code execution inside the ledger service, they still cannot rewrite a past event, because the credentials that process holds do not carry the privilege. The database refuses the statement. This turns an application-layer promise into a database-layer *capability boundary*.

Note honestly what it does *not* stop: someone who obtains the superuser or migrator credentials can rewrite anything. That is what the hash chain and the externally published Merkle roots are for. The two controls are layered deliberately — grants stop the likely attacker, hashes expose the powerful one.

### 4.4 Clock skew is recorded, never corrected

Field devices have wrong clocks. The naïve response is to overwrite the device time with the server time on arrival. Mohar stores both and the difference.

The reason is evidentiary. A device whose clock is consistently four minutes slow is a maintenance problem. A device whose clock jumped backwards by six hours immediately before a handover is a tampering indicator — and if you had normalised the timestamp on arrival, you destroyed the only evidence of it. The access engine treats skew above two minutes as a failed check, but it never edits the reported time.

### 4.5 Idempotency for offline queues

A field device in a strongroom queues events locally and uploads when it regains signal. Uploads get retried; retries produce duplicates. Each event carries a client-generated UUID, and re-submitting an ID already in the ledger returns the existing entry rather than appending a second one. Retrying is therefore always safe, which is what makes an aggressive offline retry policy acceptable.

# 5 Merkle trees and anchoring CORE

## 5.1 The limitation of a plain chain

A hash chain proves history is intact, but only if you can walk the whole thing. To convince an outsider that event 190 is genuine, you would have to hand them all 190 events. For an examination board that means disclosing every centre's custody record to prove one fact about one package. That is unacceptable.

## 5.2 The tree

A Merkle tree hashes events in pairs, then hashes those hashes in pairs, up to a single **root** that summarises everything below it.

```
                ROOT
              /      \
         H(AB)        H(CD)
        /    \       /     \
    H(A)    H(B)  H(C)    H(D)
     |       |     |       |
    ev1     ev2   ev3     ev4
```

To prove `ev3` is in this tree, you need `ev3` itself plus just two hashes — its sibling `H(D)`, and `H(AB)`. The verifier recomputes upward and checks they land on the published root. They learn nothing about `ev1`, `ev2`, or `ev4`. This is an **inclusion proof**, and its size grows as $\log_2(n)$ — proving one event out of a million costs about twenty hashes.

## 5.3 RFC 6962

Mohar follows **RFC 6962**, the Certificate Transparency specification, rather than inventing a tree format. RFC 6962 prefixes leaves with a `0x00` byte and internal nodes with `0x01`, which prevents second-preimage attacks where an attacker passes off an internal node as a leaf.

Using a published standard also means any existing CT verification library can check our proofs. "We use the same tree construction that browsers use to audit the world's TLS certificates" is a much better answer than "we wrote a Merkle tree."

## 5.4 Daily anchors and external timestamping

Each day the ledger builds a Merkle tree over that day's events and stores the root as an **anchor**, along with the first and last sequence numbers and the tree size. The anchor is the fixed point that makes tampering detectable even by someone with full database access — once yesterday's root is published somewhere outside the database, rewriting yesterday's events changes the recomputed root and the mismatch is immediate and public.

The intended external publication is an **RFC 3161** trusted timestamp, obtained free from a public authority such as freeTSA. This is a signed statement from an independent party that this exact root existed at this time.

> **State this honestly.** The anchor build and the `notarised` flag exist in the schema and the API, but the RFC 3161 client is *not* implemented in this MVP. Anchors are built and stored; they are not yet externally timestamped. Say so if asked. Claiming a notarisation you do not perform is the kind of thing that ends a review badly.

## 6 Why this is deliberately not a blockchain ADVANCED

This is recorded as `docs/adr/0001-not-a-blockchain.md`, and you will be asked about it. Have the argument ready, because "why not blockchain" is the most predictable question in the room.

### 6.1 What a blockchain is actually for

A blockchain solves one specific problem: reaching agreement on an ordered history among parties who **do not trust each other and have no shared authority**. Every expensive property — proof of work, mining, tokens, distributed consensus — exists to buy Byzantine fault tolerance among mutually hostile, permissionless participants.

### 6.2 Why that problem does not exist here

An examination board *is* a shared authority. It is legally constituted, statutorily responsible for the examination, and already the party everyone defers to. There is no set of mutually hostile peers who must agree on ordering without it. Buying Byzantine consensus you do not need means paying for throughput limits, latency, operational complexity, and — for a public exam board — either a token or a permissioned network that has all the cost and none of the decentralisation.

### 6.3 What we actually needed

| Required property | Blockchain gives it? | What Mohar uses instead |
|---|---|---|
| Tamper-evident ordered history | Yes | Hash chain — same property, no consensus |
| Attributable entries | Yes | Ed25519 device signatures |
| Third-party verification | Yes | RFC 6962 Merkle roots and inclusion proofs |
| Proof against the operator | Yes | External RFC 3161 timestamp on daily roots |
| Trustless consensus among hostile peers | Yes | **Not needed** — the board is the authority |
| Works offline in a strongroom | No | Local signing plus idempotent replay |
| Selective disclosure of one custody record | Awkward | Inclusion proof reveals exactly one event |

### 6.4 How to say it in the room

> "We use the cryptography blockchains are built from — hash chains, Merkle trees, public-key signatures — without the distributed consensus layer, because the consensus layer solves a trust problem that does not exist when a statutory examination board is the authority. We get tamper-evidence and independent verification at a fraction of the operational cost, and we work offline, which a chain requiring network consensus cannot."

Note that this also constrains the interface. The control room says "Record verified ✓ · Entry #190", never "Block #190", because the system has entries in a chain, not blocks in a blockchain. Getting the vocabulary right is part of the argument being honest.

# 7 The custody model CORE

## 7.1 The eight stages

A package's life is divided into eight named stages. Each has an expected role, and a custody key is scoped to exactly one of them.

| # | Stage | What happens | Expected role |
|---|---|---|---|
| 1 | `seal` | Authority seals the bundle and registers the seal serial | district_officer |
| 2 | `dispatch` | Package released from the authority to a courier | district_officer |
| 3 | `transit` | Courier holds the package on the road | courier |
| 4 | `custodian` | Package held in a custodian's strongroom | custodian |
| 5 | `centre` | Package received at the examination centre | superintendent |
| 6 | `unlock` | Package opened for printing — the highest-value stage | superintendent |
| 7 | `print` | Metered printing under way | superintendent |
| 8 | `destroy` | Content key zeroisation confirmed | superintendent |

## 7.2 Projection: state is derived, never stored

This is the second design decision genuinely worth defending. A package's current state — sealed, in transit, at custodian, at centre, opened, returned, compromised — is **not** a column somebody updates. It is computed by replaying every event for that package in sequence, in `services/ledger/src/domain/custody.ts`.

```
projectCustody(events) →
  { state, holderPersonId, holderRole, sealSerial,
    hops[], anomalies[], accessGranted, printed, keyDestroyed }
```

Why this matters:

- A stored state column can be wrong, or be made wrong. A derived state cannot disagree with the evidence, because it *is* the evidence.
- Adding a new anomaly detector later re-analyses all historical packages for free — no backfill, no migration.
- Every claim on screen is traceable to the specific events that produced it.

## 7.3 Divergence is the point

Each package carries two states: `declaredState`, which an operator records as the plan, and `observedState`, which the projection derives from events. When they disagree, `divergent` is true.

> **This is a feature, not an inconsistency to reconcile.** A package declared "at centre" that events show is still in transit is exactly the situation you want surfaced. Most systems overwrite the plan with reality, or reality with the plan, and lose the discrepancy. Mohar keeps both and reports the gap between them.

## 7.4 The anomaly detectors

| Code | Meaning |
|---|---|
| `illegal_transition` | The package moved between two states that no legal path connects — e.g. sealed straight to opened. |
| `handoff_holder_mismatch` | A handover claims to come from a person who was not the recorded holder. Someone handed over a package they did not have. |
| `seal_serial_changed` | The serial read at a checkpoint differs from the one registered at seal time. The strongest single indicator of physical interference. |
| `custody_gap` | A stretch longer than six hours *while in transit* with no recorded holder. |
| `printed_without_grant` | Printing events exist with no preceding granted access decision. |
| `opened_before_window` | The bundle was opened before the examination window opened. |
| `key_not_destroyed` | Printing finished but no key-zeroisation event followed. The content key is still live. |

> **A bug worth mentioning as evidence of care.** The custody-gap detector originally flagged any long silence, which meant a package sitting legitimately in a custodian's strongroom overnight was reported as a gap. It now only counts silence while the package is `in_transit`. A detector that fires on correct behaviour trains operators to ignore it, which is worse than not having it at all.

# 8 Rotating custody keys ADVANCED

## 8.1 The idea

Every stage of every package requires a **custody key** — a short code the authority at that stage must present. A key is valid for exactly one six-hour **epoch** and is scoped to one package and one stage.

```
EPOCH_SECONDS       = 6 * 60 * 60   // 21600
EPOCH_GRACE_SECONDS = 30 * 60       // 1800

epoch = floor( unixSeconds / 21600 )
```

Four epochs per day, aligned to fixed boundaries. Everyone in the system agrees on the current epoch without coordinating, because it is a pure function of the clock.

## 8.2 Expiry by arithmetic, not by a job

This is recorded as `docs/adr/0005-custody-keys-expire-by-arithmetic.md` and it is the sharpest security argument in the project.

The obvious implementation is a scheduled job that marks keys expired. Consider its failure mode: the scheduler dies at 02:00, nobody notices, and every key in the system stays valid indefinitely. The failure is silent and it **fails open** — the worst possible direction for an access control.

Mohar has no expiry job. A key's epoch is compared against the current epoch at the moment of the request:

```
if (keyEpoch !== currentEpoch) → deny, reason key_expired
   evidence: "presented epoch 76891, current epoch 76894"
```

> **The consequence.** If every background process in the system dies, keys do not stop expiring — because nothing was ever making them expire. Time makes them expire. Broken infrastructure denies access rather than granting it. The system **fails closed**.

## 8.3 The grace window

A thirty-minute grace at the epoch boundary handles the real case where a superintendent receives a key at 05:58 and presents it at 06:02. Without grace the system would deny a completely legitimate act because a clock ticked over mid-procedure, and the operational response to that would be to widen epochs to twenty-four hours, which destroys the whole property.

## 8.4 The key format

```
MHR-UNLOCK-8K3M-P7QW-2NRT
```

The random portion uses **Crockford base32**: the alphabet `0123456789ABCDEFGHJKMNPQRSTVWXYZ`, which deliberately omits **I**, **L**, **O** and **U**.

- **I** and **1**, **O** and **0**, **L** and **1** are confusable when read aloud or handwritten. This key gets dictated over a bad phone line from a rural centre.
- **U** is excluded so no random draw produces an obscene string. This matters in a government deployment.
- Uppercase only, grouped in fours, so it is transcribable without error.

The random payload is 128 bits, which is far beyond brute force, and the format is chosen entirely around a human reading it correctly under stress.

### 8.5 The key is never stored

```
create table led.access_key (
  ...
  key_hash    bytea not null,    -- SHA-256 of the key
  fingerprint text  not null,    -- short prefix, safe to display
  unique (package_id, stage, epoch)
);
```

The plaintext key is shown exactly once, at issue. After that only its SHA-256 exists. A full database dump yields no usable keys — handled precisely the way a password would be. The `fingerprint` is a short non-secret prefix so the control room can say *which* key was presented without being able to present it.

Verification uses a **constant-time** comparison, so an attacker cannot learn the key one character at a time by measuring how long a rejection takes.

The `unique (package_id, stage, epoch)` constraint means issuing is idempotent: asking twice in the same epoch returns the same key rather than minting a second valid one.

## 9 The access decision engine ADVANCED

`services/ledger/src/domain/policy.ts` — the function that decides whether a bundle may be opened. If you learn one file in this repository, learn this one.

### 9.1 Deny by default

A request begins refused. There is no path through `decideAccess` that returns `granted` without every one of fifteen checks having positively passed. Grant is the exception that must be earned, not the default that failures subtract from.

### 9.2 The fifteen checks

| Check | What it establishes |
|---|---|
| `key_presented` | A custody key was supplied at all |
| `key_valid` | Its SHA-256 matches an issued, unrevoked key |
| `key_in_window` | Its epoch is the current epoch, within grace |
| `key_stage_match` | It was issued for this stage and this package |
| `device_enrolled` | The signing device is known and not revoked |
| `device_binding` | The device is bound to this centre |
| `roster_membership` | The person is on the roster for this centre |
| `role_permitted` | Their role is the one this stage expects |
| `geofence` | Reported position is inside the centre's radius |
| `geo_accuracy` | The GPS fix is accurate to within 50 m |
| `custody_window` | The request falls inside the permitted time window |
| `clock_skew` | Device clock is within two minutes of the server |
| `seal_serial` | The serial read matches the one registered at seal |
| `package_state` | The package is not compromised or already opened |
| `exam_active` | The examination has not been suspended |

### 9.3 Every check runs, always

The engine never short-circuits on the first failure. This costs a few comparisons and is deliberate:

> An attempt that trips *four* checks is a different event from one that trips a clock skew. A courier at the wrong centre, outside the window, with a stale key, presenting the wrong seal serial is a coordinated attempt; a superintendent whose phone clock drifted is an operations issue. If you short-circuit you record the first failure and lose the shape of the attempt — and there is no second chance to observe an attempt that has already happened.

### 9.4 Evidence, not verdicts

Each check records what was *observed*, not what it concluded:

| What a naïve system stores | What Mohar stores |
|---|---|
| `outside_geofence` | `"412 m from centre boundary; geofence 150 m"` |
| `key_expired` | `"presented epoch 76891, current epoch 76894"` |
| `clock_skew_excessive` | `"device clock 4 min 11 s behind server"` |
| `seal_serial_mismatch` | `"registered MH-4471-A, read MH-4471-B"` |

A verdict cannot be re-examined; its inputs are gone. Evidence can be re-examined years later under a different policy, and — this is the operative point — it can be annexed to a First Information Report. "412 metres from the centre" is a fact a court can weigh. "outside_geofence" is a computer's opinion about a fact it did not keep.

### 9.5 The attempt is recorded before the caller is told

The write to `led.access_attempt` happens **before** the response is returned. An attacker cannot learn the outcome and then kill the connection to prevent the record. And because `led.access_attempt` carries only `SELECT` and `INSERT` grants, the attempt cannot be deleted afterwards either — not by the application, not by anyone holding its credentials.

Denied attempts are the most valuable records in the entire system. They are the ones that describe someone trying.

### 9.6 The moment the engine proved it was real

> During development the seed script produced a run in which the engine refused *every* centre, citing `person_not_on_roster` and `outside_custody_window`. The cause was a bug in the seed data — the scenario timeline was set six hours in the future, outside every custody window.
>
> This is worth telling in the demo. A simulator that asserts its own outcomes cannot catch its author's mistake. Ours did. The engine was evaluating real conditions against real reference data and refusing them, which is exactly the behaviour you want and exactly what a scripted mock cannot demonstrate.

## 10 Why the severity labels were removed

### 10.1 What was there before

An earlier build had an alarm queue tagging each event `CRITICAL`, `HIGH`, `MEDIUM`, `LOW`. It looked like a security product. It was deleted on purpose.

### 10.2 Why it was wrong

- **The label is invented.** Nothing in the evidence says a seal mismatch is "critical" and a clock skew is "medium". Someone chose that, then the interface presented the choice as though the system had measured it.
- **It replaces the facts on screen.** Operators read the badge and stop reading. The colour becomes the finding, and the 412 metres never gets looked at.
- **It trains people to ignore things.** After a week of amber banners that turn out to be nothing, amber means nothing.
- **It does not survive scrutiny.** In a court or an inquiry, "the system rated it critical" invites the question "on what basis", and there is no answer. "The device was 412 metres outside the geofence at 04:11" needs no defence.

### 10.3 What replaced it

The **Activity ledger**. Every recorded act appears with its facts — who, which device, which key and its fingerprint, epoch presented against epoch current, position and distance, clock skew, seal serial read, the verbatim payload, and the chain hash.

The only judgement the system retains is two fields:

| | |
|---|---|
| `requiresDecision` | A boolean. Does a human still need to act on this? Not how bad it is — whether it is *open*. |
| `consequence` | A sentence from the runbook describing what follows operationally. |

That is a defensible line. The system says "this is unresolved and here is everything known about it." The human, who has context the system does not, decides what it means.

# 11 Repository map and how to run it

## 11.1 Layout

```
codeaxis/
├── apps/
│   ├── control-room/     React + Vite operator console  ← the demo UI
│   ├── centre-client/    examination-centre application
│   ├── field-app/        courier / custodian device app
│   └── verify-portal/    third-party verification surface
├── services/
│   ├── ledger/           Fastify API — the system of record ← the core
│   │   └── src/domain/   policy.ts · custody.ts · keys.ts · activity.ts
│   ├── access/  gateway/  notify/  render/  sealkeys/  trace/  unlock/
├── packages/
│   ├── crypto-core/      hashing, Ed25519, JCS, custody-key.ts
│   ├── contracts/        zod schemas — the shared event vocabulary
│   ├── ledger-client/    typed client
│   └── ui-kit/
├── infra/migrations/     001_init.sql · 002_custody_keys.sql
├── tools/seed/           drives the real engine to produce demo data
└── docs/                 00-11 design docs + adr/
```

## 11.2 Where each concept lives

| | |
|---|---|
| Hash chain | `packages/crypto-core` and `services/ledger/src/store/` |
| Canonical JSON | `packages/crypto-core` (`canonicalize`) |
| Custody keys / epochs | `packages/crypto-core/src/custody-key.ts` |
| Access decision engine | `services/ledger/src/domain/policy.ts` |
| Custody projection | `services/ledger/src/domain/custody.ts` |
| Activity ledger | `services/ledger/src/domain/activity.ts` |
| Append-only enforcement | `infra/migrations/*.sql` — the `grant` statements |
| Timeline UI | `apps/control-room/src/pages/PackageDetail.tsx` |

### 11.3 Running it

```
# prerequisites: Node 20+, pnpm 9, PostgreSQL 18 running locally

pnpm install
pnpm build

# schema — run as the migrator role, which alone holds DDL rights
MIGRATE_DATABASE_URL=postgres://mohar_migrator:dev_only_password@localhost:5432/mohar \
  pnpm migrate

# ledger API — runs as the app role: INSERT + SELECT only
DATABASE_URL=postgres://mohar_app:change_me_in_deployment@localhost:5432/mohar \
  node services/ledger/dist/index.js          # → http://localhost:8081

# demo data — this drives the real engine, it does not assert outcomes
pnpm --filter @mohar/seed start

# operator console
cd apps/control-room && npx vite               # → http://localhost:5173
```

> **Note on port 8081.** Opening it in a browser returns `{"message":"Route GET:/ not found"}`. That is correct — the ledger is an API with no root route. Try `/health` or `/summary`.

### 11.4 The five seeded scenarios

The seed tool issues real keys, calls the real access engine, and emits signed events based on whatever the engine actually rules. It does not fabricate outcomes.

| Package | Scenario | What to point at |
|---|---|---|
| `JPR-001` | Clean run, but the content key was never zeroised after printing | `key_not_destroyed` — an unresolved act with no severity attached to it |
| `JPR-002` | Eighteen-hour transit gap | `custody_gap` — nobody is recorded as holding the bundle overnight. The silence is the finding. |
| `JPR-003` | Seal serial read differs from the registered serial | Evidence stored as both serials, not as the word "mismatch" |
| `JPR-004` | Key three epochs stale | Denial evidence reads "presented epoch N, current epoch N+3" — expiry by arithmetic, demonstrated |
| `JPR-005` | Unissued key, presented from outside the geofence | Multiple checks fail in one attempt — the value of never short-circuiting |

### 11.5 A demo that lands in five minutes

1. **Overview** — what is moving, what was refused, what nobody has resolved.
2. **Workflow → JPR-002** — walk the timeline. Under each event, the full record: actor, device, position, clock skew, body hash, chain hash, verbatim payload. Stop on the gap.
3. **Activity** — show a denied attempt. Read out the evidence, not a label.

4. **Keys** — the epoch bar counting down. "These rotate in 43 minutes. Nothing rotates them — they expire because time passed."
5. **Integrity** — recompute the chain live. Then say what it would mean if it did not verify.

# 12 Honest gaps and likely questions

## 12.1 What is not built

> Know these before a judge finds them. Being first to name your own gaps is worth more than any feature you could add tonight.

| Gap | Status |
|---|---|
| Authentication on the ledger API | Not implemented. The `gateway` service is where it belongs and is currently a stub. The API is unauthenticated in this MVP. |
| RFC 3161 timestamping client | Not implemented. Anchors are built and stored; the `notarised` flag is never set true. |
| Device attestation verification | Devices are enrolled but hardware attestation is not verified. A forged device record would be accepted. |
| Rate limiting on `/access/request` | Absent. Key guessing is infeasible at 128 bits, but the endpoint is trivially floodable. |
| Shamir 3-of-4 secret sharing | Designed in `docs/03-crypto-design.md`; not wired into the running unlock path. |
| drand / tlock time-lock encryption | Designed — encrypt to a future beacon round so a paper is undecryptable before its time. Not implemented. |
| Git history | The repository has no commits. Every revert so far has been a hand reconstruction. This should be fixed before the presentation. |

## 12.2 Questions you will be asked

### Why not blockchain?

> Blockchain buys consensus among mutually distrusting peers. An examination board is a statutory authority — that trust problem does not exist. We use the cryptography blockchains are built from, without the consensus layer, and we work offline, which a consensus-dependent chain cannot. See §6.

### What stops your own administrator from editing the record?

> Nothing stops them; everything exposes them. The application's database role has no `UPDATE` or `DELETE` grant, so a compromised service cannot do it at all. A superuser can — and doing so breaks every chain hash after the edited row and invalidates the daily Merkle root already published outside the database.

### Does this prevent leaks?

> No, and we do not claim it. A superintendent with legitimate access can photograph a paper. What we change is the window — days to minutes — and the suspect set — hundreds to a named few, with evidence.

### What happens when the network is down?

> Devices sign and queue events locally and upload on reconnection. Every event carries a client UUID and the ledger is idempotent, so retries never duplicate. Access decisions that need server state fail closed — denied, and recorded as denied.

#### What if your key-rotation service dies?

> There is no rotation service. Keys expire because `floor(now / 21600)` advances. If every background process in the system dies, keys still expire — broken infrastructure denies access rather than granting it.

#### Why no severity levels on your alerts?

> Because the severity is invented and the facts are not. We show the distance, the epochs, the skew, the serials, and one boolean — whether a human still needs to act. A label a court can question is worse than a measurement it cannot.

#### Why store both a declared and an observed state?

> Because the disagreement between plan and reality is the finding. Systems that reconcile the two lose the only signal that mattered.

#### Is your demo data real or scripted?

> The seed tool issues real keys, calls the real access engine, and emits events based on whatever it actually rules. It caught a genuine bug in our own seed timeline by refusing every centre — a script that asserts its outcomes cannot do that.

#### What does this cost to run?

> The stack is Node, Postgres, and free public services only — no HSM, no paid API, no cloud dependency. This is recorded as an architecture decision ( `adr/0004-no-paid-dependencies.md` ), because a per-centre licence cost is what kills deployment at district scale.

### 12.3 The three sentences to have ready

> **What it is.** "Mohar is a tamper-evident custody chain for examination papers — every handover is signed by the person who performed it and written into an append-only ledger that nobody, including us, can edit without it being visible."
>
> **Why it matters.** "Exam-fraud cases are frequently detected and rarely convicted. That is not a detection failure — it is an evidence failure. We produce custody evidence that survives cross-examination."
>
> **What it does not do.** "It does not make examinations leak-proof. It makes leaking expensive, narrow, and attributable."

---